

Halo communication in the mEVP sea-ice solver of the FESOM2 GPU port

N. Koldunov, 6 August 2026 — summary of measurements

Three ways of exchanging the halo less often

mEVP advances the ice momentum in 120 sub-cycles per model time step, and the ice-velocity halo is at present exchanged after each of them. We implemented and measured three alternatives.

Delayed exchange — exchange every K -th sub-cycle and accept a halo up to $K - 1$ sub-cycles old. This is the largest saving of the three, but it leaves the sub-domain boundaries visible in the ice field. *Wide halo* (Danilov) — exchange every K -th sub-cycle, and recompute a K -deep ring of ghost cells in between so that no value is stale; the result reproduces the every-sub-cycle trajectory bit for bit. *Divergence form* (Danilov) — carry $\nabla \cdot \sigma$ at nodes rather than σ at elements; it agrees with $\nabla \cdot \sigma$ to 10^{-15} relative, and its effect on run time differs between the two backends.

Cost, measured against the model step and against the ice

Each number is given twice. Sea ice is 5 to 13% of a CPU model step, so an ice-only change is divided by about ten before it reaches the step, and the share of the ice cost (ice, ice dynamics and ice advection, computation together with time spent waiting for communication) is the measure appropriate to a sea-ice scheme.

		GPU		CPU	
		ice cost	model step	ice cost	model step
delayed exchange	$K=2$	−32 to −39%	−8.3 to −9.3%	−2 to −18%	−0.1 to −2.9%
delayed exchange	$K=4$	−37 to −58%	−8.5 to −14%	−5 to −25%	−0.3 to −3.6%
wide halo, standard form	$K=8$	−11 to −24%	−2.2 to −5.7%	never profitable	
wide halo, divergence form	$K=8$	+5 to −29%	+0.2 to −6.8%	never profitable	
wide halo, either form	$K=2$	−15% [†]	+1.7 to −3.8%	never profitable	
wide halo written as widened loops	$K=8$	−37 to −58%[‡]	−7.8 to −14.4%	not profitable	
divergence form alone	—	+2 to +17%	0 to +1.6%	−2 to −8.5%	−0.5 to 0%

GPU ranges span the meshes at the node count where each still gains from more GPUs: CORE2 on 1 node (4 GPUs), fArc on 4, DARS on 8, NG5 on 16 — the wide-halo rows cover the first three, NG5 being still in the queue. CPU ranges span 128 to 2048 cores on CORE2 and fArc. [†] one point only (DARS, 8 nodes); at $K=2$ the wide halo is a cost on fArc at 4 nodes and a saving at 16, so it needs the larger K . [‡] CORE2, fArc and DARS; NG5 still in the queue. The range spans the meshes and the two forms, the divergence form leading at all three.

On CPU the saving grows with core count (−5% of the ice at 128 cores, −25% at 2048); on GPU it falls as each GPU holds more of the mesh (−58% at 32 000 surface nodes per GPU, −37% at 460 000). The divergence form removes 2 to 8.5% of the ice cost from 512 cores upward, and is invisible in the model step only because of the denominator. On GPU it costs, because the sub-cycle there is limited by how often work is dispatched to the device rather than by arithmetic or memory traffic, so removing element arrays removes work that was not being paid for. Danilov’s ~20% for the divergence form with its double halo corresponds to our −19.1% of the ice cost for the same combination (fArc, 16 GPU nodes).

Why the delayed exchange is not usable

Under the delayed exchange the stale values lie on the one-cell boundary of each sub-domain, and the decomposition is fixed for the run, so the same nodes are perturbed at every step. On fArc with 16 GPUs over 60 days, $|\Delta \text{ ice thickness}|$ averaged on those boundaries is 2.5 times its interior value at $K=4$ and 1.7 times at $K=2$, against 0.85 for two identical runs and 0.87 for the exact wide halo. The difference is concentrated there from the first day and appears on maps as a network following the sub-domain edges. A polygonal structure in the ice field, in the shape of the domain decomposition, is not acceptable in a sea-ice model, and that ends the matter — the scheme’s accuracy in the integral diagnostics is beside the point. Section 6.6 and Figure 4 of the full note give the profiles, the map and the controls.

Recommendation

For production we would use the wide halo at $K=8$ on GPU, in the divergence form, written as a widening of the loops that already exist rather than as ring kernels of their own. That single change takes the saving from 10–28% of the ice cost to 37–58%, and it reverses which form is better: with ring kernels the standard form led on the two smaller meshes, with widened loops the divergence form leads on all three. Its cost is bit-identity: a ring node is summed in the receiving process’s element order rather than the owner’s, which after a time step moves the ice velocity by 8 to 15 units in the last place, and the 60-day decomposition test passes. Where bit-identity is required the kernel-per-ring version remains and removes 10 to 28%. On CPU the wide halo is not profitable in either writing, on any mesh we tried.

We do not regard the delayed exchange as usable: it places the domain decomposition in the ice field. We had also recorded that it saves about three times as much, and that is no longer true — in one allocation on CORE2 the widened-loop wide halo is identical to it in both currencies while leaving no imprint, and on fArc and DARS it closes about two thirds of the gap.

The divergence form should be judged in two roles. On its own it costs 2 to 8% of the ice on GPU and saves 2 to 8.5% on CPU at large core counts. As the carrier of the wide halo, **how it compares with the standard form depends entirely on how the ring is written**, and this is the result that changed most. With ring kernels of their own it lost by 13 percentage points of ice cost on CORE2 and 4 on fArc and gained 4 on DARS. Written as widened loops it **leads on all three** — by 1.3, 6.5 and 6.7 points, ordered by how much mesh each GPU holds, and at every K tried. The ring kernels were not measuring the reformulation; they were burying it under a launch cost that fell hardest on the divergence form, whose ring gather was the most expensive of them. NG5, at 116 000 nodes per GPU, is queued and will test whether the ordering continues.